

Exercise 1: Setting Up and Getting Started

1. Using Octave in the Griffin Research Group

Octave is an open source programming language / programming environment that we use for computational chemistry. While commercial software programs can provide some of the functionality we need, writing our own computer programs gives us complete control. Because we are writing the programs ourselves, we can make the computer behave exactly as we want it to. Octave is a high-level language, meaning that it is quite user friendly, once you grow accustomed to it. It is also open-source, meaning that it is freely available on the web. The benefit of learning to use open-source programming tools is that you can continue to use them when you leave DePaul, in either personal or professional settings, without needing to purchase any commercial software.

The only drawback is that, when problems arise, support resources are somewhat limited. There is no 24 hour “tech support” to call when a problem comes up, although helpful tutorials, wiki sites, and message boards are readily available on the internet. It will be up to you to figure out how to make your programs work, albeit with some guidance from Dr. Griffin. The primary skills you must develop to become a good computer programmer are identical to the skill set you will need to be a good scientist: willingness to take on a task you do not completely understand, courage to pick a hypothesis and test it out, and tenacity to keep trying stuff until something works.

2. Setting Up Octave

Octave was developed for the GNU/Linux operating system. It has since been ported to Mac and PC systems. Files and instructions for downloading and installing Octave can be found here:

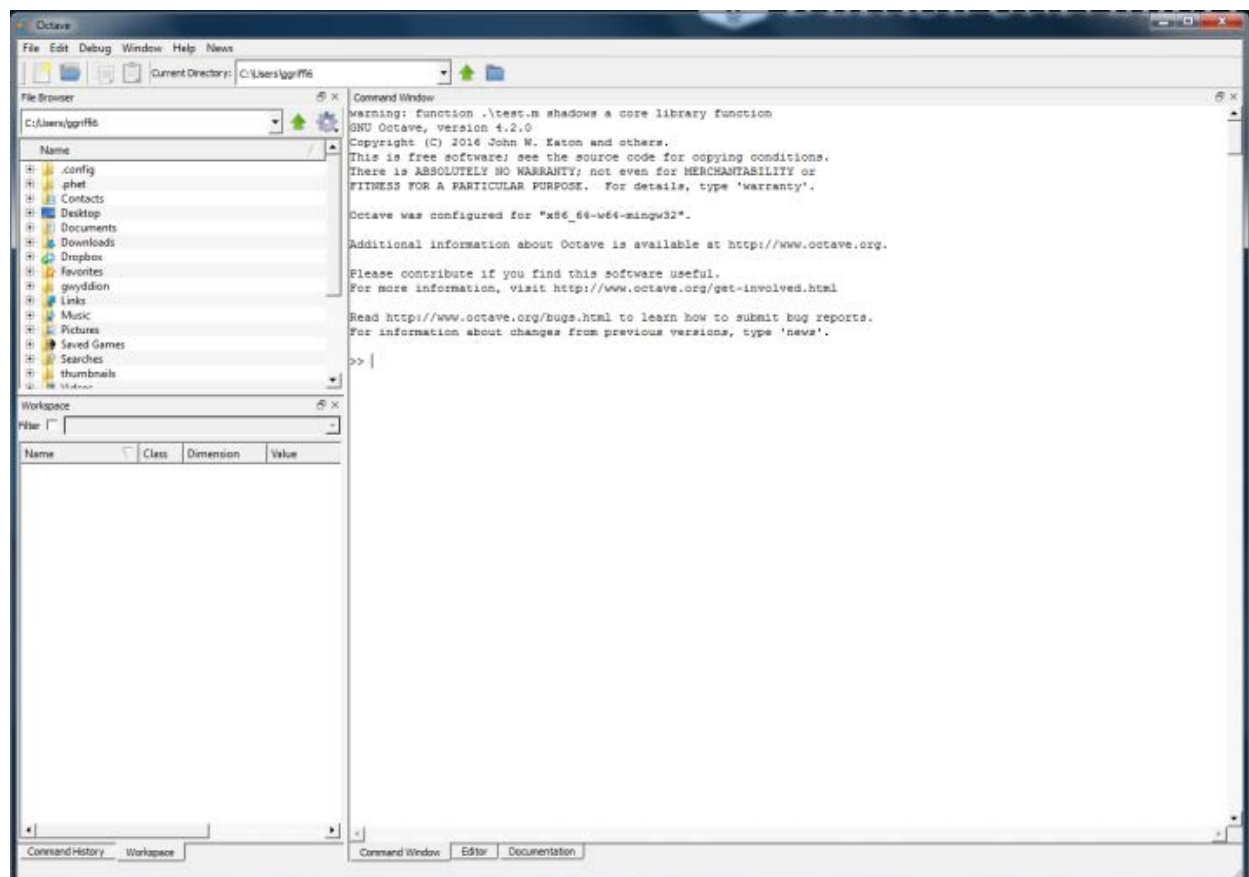
<https://www.gnu.org/software/octave/download.html>

The best version of Octave to work with is usually the most recent. Make sure you select a version at least 4.0 or later so that you will have access to the graphical user interface (GUI). The GUI is a convenient piece of software that will help you make and run your programs.

Your first task is to get Octave installed on the computer you would like to use. If you want to use the computers in the research lab, they should already have Octave loaded. If you are using your own computer, use the website above to find the appropriate installation software for your operating system. Follow the directions on the website to get Octave installed, and consult Dr. Griffin if you get stuck or have questions. The process is simplest for Windows computers, you will click on a link and download an installer program. Run the installer program and you should soon have some Octave shortcuts on your desktop. Mac instructions are a bit trickier, there are several methods and the best choice depends on your operating system. Read the instructions carefully and give it a shot! Octave is also available on several versions of Linux. To get instructions for your specific version of Linux, google “install octave [version]”. Write down the file path for the directory Octave is installed to, you will want this information later.

3. The Graphical User Interface, and Your First Computer Program

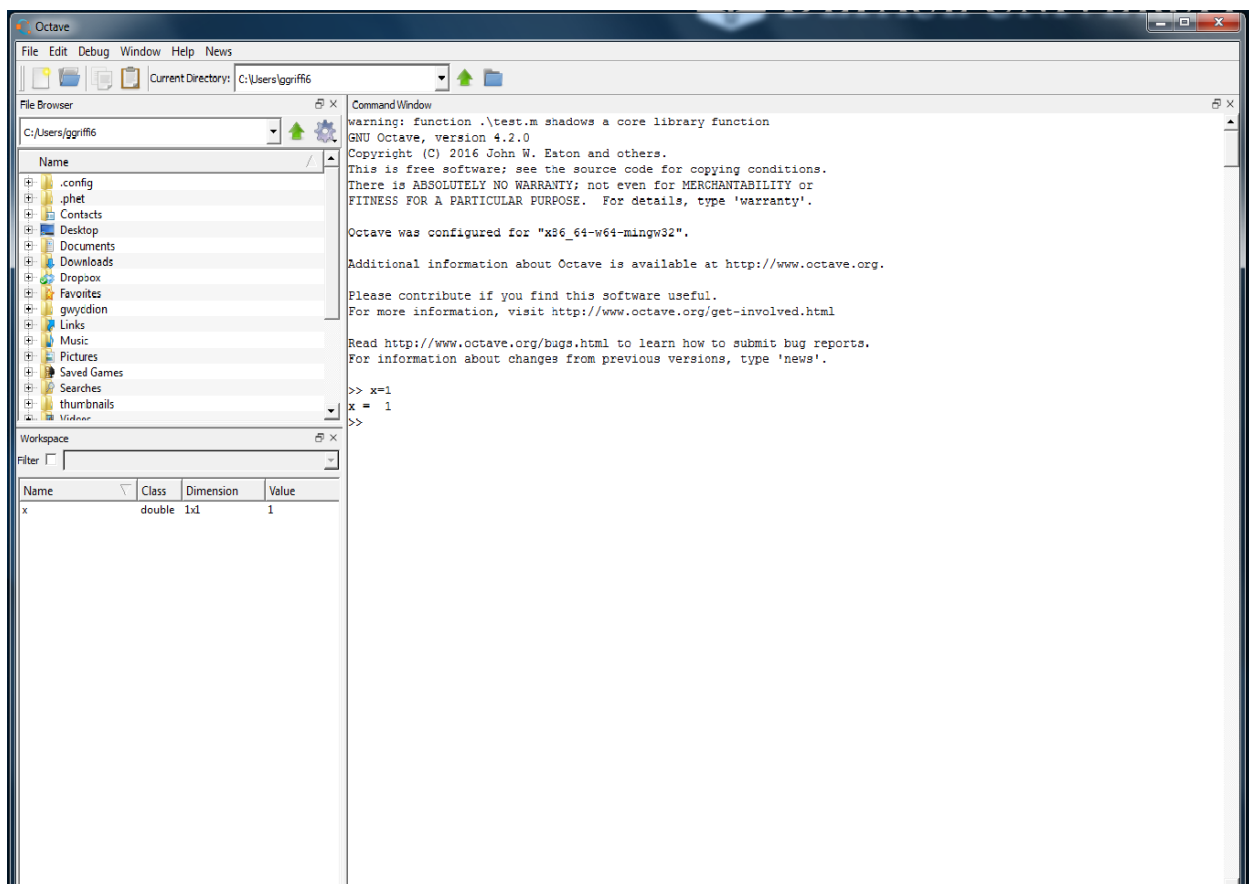
Open the Octave GUI shortcut. The GUI is our software development environment. It should look like this:



3.1 The Command Window

The big panel on the right is our Command Window interface. If we want to try out commands one at a time, we can type them into the Command Window and hit the enter key. Every time we hit enter, the computer will read our text and take actions based on the rules of the Octave programming language. The other major parts of the GUI are the File Browser (top left) and the Workspace (bottom left). We'll deal with these more later. Briefly, the File Browser helps us find and manipulate files on our computer, while the Workspace lists and organizes all the information we've asked the computer to remember during one of our programs.

Type in the text "`x = 1`" into the prompt in the Command Window and hit enter (the quotation marks are not part of the command text.) Congratulations, you have just written a computer program! Your Octave GUI should now look like this:



We can see several important changes. In the Command Window, we can see your command text, “`x = 1`”. Immediately below that, we see output from the computer, the text “`x = 1`”. We can also see that there is now an entry in the Workspace box, the lower left part of the GUI.

Here’s what happened. When you hit enter, the computer read your command text and interpreted it according to the rules of the Octave programming language. In Octave (and pretty much every other programming language), the `=` sign is used to assign values to variables. The computer reads the command text and interprets it as a sequence of instructions:

1. Create a variable called `x`
2. Assign the value of 1 to the variable `x`.
3. Report the value of `x`

We’ll get into how computer programs work, what variables are, and what it means to assign a value to a variable, in future exercises. For now it is enough to understand that the computer has read your command text, and followed a set of rules to carry out a specific sequence of actions. The set of rules that translates your English words and symbols into tasks for the computer to carry out is the Octave programming language. Using this tool, we can instruct the computer to carry out any sequence of actions we want. All we need to figure out is instructions we need, and what order to put them in. This is the basis of every computer program you have ever used, whether it was a time-killing game on the internet or the software that runs your bank ATM. They are all just computers executing a sequence of instructions.

When we said earlier that Octave is a high-level language, what we meant is that the commands we give it will look something like written English. This makes the program easier to read and think about for us humans. Computers think in pieces of binary data called bits, which you can picture as tiny boxes that each contain either a zero or a one. Inside the computer, every instruction we give will be broken down into these bits before being executed by the computer’s processor. By working in a high level language, we avoid having to look at the information at the level of bits. The only cost is that the computer has to translate our English words into the appropriate bits, which takes a little bit of time. This process is called “compiling” the program.

Fortunately, for the small programs we use around the lab, compiling doesn't take long enough to be a problem.

Above we specified that the computer executed three tasks for us: creating a variable, assigning a value to that variable; and reporting the result. This third task is often not needed, because we often want the computer to do a few things in a row before reporting a value back to us. To suppress the output associated with a given command, we can put a semicolon at the end of the line. Type `"a = 2;"` into the Command Window and press enter. If you look in the Workspace, you will see that the computer has executed your command and created a variable `a` that is assigned a value of 2. However, the computer did not report back `"a = 2"`.

3.2 The Workspace

The bottom left portion of the GUI is the Workspace. The workspace displays all the information we have asked the computer to remember so far. Right now it tells us that we have defined two variables named `x` and `a`. It then lists the Class, Dimension, and Value of each variable. We will now work on deciphering what all that means.

We said above that by typing `"x = 1"` into the command window and pressing enter, we instructed the computer to create a variable called `x` and assign it a value of 1. This means that the computer designated a portion of its memory (RAM) to hold the value of `x`, then adjusted the bits within that portion of the memory so that they record the value 1. We could go into lots more detail about this (more about bits and how computers store, read, and interpret them), but for our purposes it is sufficient for us to know that the computer is now going to remember two things:

1. a specific portion of the computer's memory is designated to hold a value that we can refer to by typing `x`
2. the value stored in that memory is currently 1

We can change the value associated with the variable `x` at any time. Type `"x = 3;"` into the command window. If you check the Workspace, you will see that the value associated with variable `x` has been changed to three. The computer has gone to the memory portion that stores the value of `x`, and changed it so that the value is now 3.

The Workspace defines the Class of both of our variables as ‘double’. This is short for “double precision floating point”, and what this designation tells us three things:

1. how big the portion of memory allocated to each variable is
2. how the computer will interpret the data stored there
3. how big the number stored there can be

Again we could go into all kinds of detail about the mechanics underlying this information (the bits again), but we don’t really need to for now. Octave will define all of our variables as doubles by default. Octave will also read our variables as numerical values stored in the double format by default. If we want it to do something else we will have to create specific commands. This works to our advantage most of the time. The maximum values for a double are $\pm 1.7 \times 10^{308}$, and the values are specified to a precision of 15 digits. This is large enough that it almost never becomes a problem.

The Workspace next specifies the Dimension of our variables. Both `a` and `x` are specified as 1x1, meaning they contain only one value. Variables can be created containing multiple values organized into structures called vectors, matrices, or tensors. This structure is very useful to us as programmers, and we will learn all about it during this training sequence.

The final characteristic of our variables shown in the workspace is the Value. For our simple one-value variables, these are simply the value currently stored for each variable.

If we want to remove variables from the computer’s memory, we can do so at any time. We use the “clear” command. Type “clear x;” into the Command Window and press enter. You should see the variable `x` disappear from the workspace. If you only type “clear”, Octave will eliminate all variables from the Workspace.

3.3 The File Browser

The top left box of the GUI is the File Browser. This area shows us the contents of the current working directory for Octave. Large programs often need to call on other files to do their jobs, so Octave needs to keep track of a “home base” file directory. Octave can navigate out of this

directory to find files it needs, or we can gather all the necessary files into one home directory for our program.

To stay organized, you should create a directory on the computer for your programming work. To do this, navigate to your desktop using the File Browser (single click to expand a folder, double click to open it). Then click on the gear in the upper right corner of the File Browser, and create a new directory with an appropriate name, something like “Programming Tutorial” or “Octave Work” or whatever you like. Within this directory, you can further organize your programs by gathering them into folders of related files if you wish.

It will be useful to have Octave start up with the File Browser set to this folder. This will save you the time required to click through to your Octave folder every time you do some programming. To set this up, highlight and copy the file path for your Octave working directory. Make sure your file path goes all the way back to “C:\ ...” (or whatever drive you are working on, C is the default). Now right click on the Octave GUI desktop icon, and click Properties. There will be a “Start In” field. Type (or paste) the file path for your working directory. Close Octave and start it up again. If you look at the file browser (the top left-hand box), it should show that Octave is looking in your working directory.